Practical 6 - Demo Doc
Wonder Team

Miss. Keagan van Biljon - **u23594412**
Mr. Matthew Malan - **u25009801**
Mr. Lucian van der Merwe - **u25054262**
Miss. Hayley Nel - **u25101821**
Mr. Ruben van der Vyver - **u25007786**

Github Repository

Workflow research:

A **workflow** is a is a series of repetitive, tedious tasks and manual processes

A **Workflow Management System** is software that automates some or all of the tasks in a workflow, with the aim of:

- Improving productivity
- Freeing up resources (personnel etc) to be used elsewhere

Work items represent individual tasks within a workflow to be used or worked on by multiple processes, and have their own properties. They are assigned and routed by use of the **Strategy Pattern** that allocates work items based on whatever strategy is needed.

Workflow systems must also manage the different **states** of a work item or process. For example, an approval task may be approved, rejected or returned for further action. These outcomes determine how the workflow proceeds.

Participants are the individuals responsible for performing tasks or making decisions within a workflow.

Roles define their responsibilities and permissions.

Some workflow management systems use **role-based** access control or model their users in a **hierarchy**. A user can have privileges to access certain workflows or even different parts of the software to determine which users can view, create, edit or approve particular processes. For example, Interfacing provides role management and responsibility allocation, as well as automated review and approval cycles. This demonstrates the importance of incorporating permissions and participant responsibilities into a workflow system.

Existing Workflow Management Systems

Several existing workflow management systems provide useful examples of features that could be incorporated into a reusable workflow engine.

Nifty (Project Management) provides integrations with external applications to support communication and the transfer of information between systems. For example, its GitHub integration allows commits, merges and pull requests to be tracked within task activity feeds. Nifty also supports integrations through APIs and webhooks, allowing organisations to connect external applications to their workflows. This demonstrates how a workflow management system can communicate with external services without requiring all functionality to be implemented within the system itself.

Sensus Process Management provides version control for process models. Users can maintain multiple versions of a process and compare them to identify changes, additions and removals. It also provides a central process library for managing multiple process models and synchronising information between connected models. Additionally, it supports role-based permissions to control which users can model, edit or publish processes. These features demonstrate the importance of process versioning, centralised management and access control.

Trello provides reusable board and card templates that allow users to replicate existing workflows. Instead of creating a new workflow from scratch, users can create a board from an existing template, retaining its predefined structure and content. This is particularly useful when organisations need to implement similar workflows across multiple projects.

Hierarchical and Nested Workflows

Workflow systems may also support hierarchical structures, where a larger process is divided into **smaller subprocesses** or activities. This allows complex workflows to be broken down into manageable sections while maintaining an overall structure.

In a hierarchical workflow, an outer process may **depend** on the completion of an inner subprocess before it can continue. If the subprocess cannot complete successfully, the workflow may enter an exception or waiting state, allowing a participant to review the situation and determine the next course of action. This approach allows complex processes to be managed without losing control of the overall workflow.

Works Cited

doforms.com. “Doforms Project Management.” *doForms: Mobile Forms Software for Business Forms and Data Collection*, http://doforms.com. Accessed 1 November 2026.

interfacing. “Interfacing Buisness Project Managment.” *Interfacing*, https://interfacing.com/business-process-management. Accessed 1 November 2026.

nifty Project Management. “Niftypm.” *Nifty™ | The Ultimate All-in-One Hub for Every Project*, http://niftypm.com. Accessed 1 November 2026.

Sensus Process Management. “Sensus Process Management.” *Sensus process management: Home*, https://sensus-processmanagement.com. Accessed 1 November 2026.

Trello. “Trello.” *Capture, organize, and tackle your to-dos from anywhere*, http://trello.com. Accessed 1 November 2026.

Assumptions

The design and implementation of TIVIDY are based on several assumptions regarding the operation of the workflow engine and the environments in which it will be used.

1. **Predefined workflows:** Workflow definitions are created and configured before execution. The engine is responsible for executing these definitions rather than dynamically generating new processes.
2. **Registered participants:** All participants involved in a workflow are identifiable within the system and have associated roles or authority levels. These roles determine which work items participants may access or approve.
3. **Reusable architecture:** Different industry workflows can be represented using the same underlying workflow engine, with industry-specific behaviour implemented through configurable workflow definitions and extensible components.
4. **Valid input:** Required information supplied when creating workflow instances or completing work items is assumed to follow the expected data formats and validation rules.

TIVIDY Boundaries

- It does not physically complete a task eg. in scenario 2; the pharmaceutical products are delivered, but not by the system
- TIVIDY does not make professional decisions eg. Rules for approval or assignment

Industries (Scenarios)

Industry 1: Video Game Development – Copper Finch Interactive

Process: The workflow manages the development and release of The Sunken Crown content update. It starts with feature approval, followed by parallel development in the art, gameplay, audio and narrative departments. Their work is integrated into a build, tested by QA, approved by management and submitted for platform certification before release.

Participants: Executive Producer, Production Lead, artists, animators, gameplay and tools engineers, audio designers, narrative designers, integration and build engineers, QA testers, Creative Director, platform specialists and Release Manager.

Branching: The workflow uses parallel execution for the four development departments and the three platform certification processes (Steam, PlayStation and Nintendo). Approval checkpoints determine whether the workflow can progress.

Failure paths: Failed QA tests result in defects being assigned to the relevant department for correction and retesting. Failed builds require investigation and rebuilding, while rejected platform certifications require corrective work and resubmission.

Outcomes: The update is successfully released once development, testing, approvals and platform certifications are complete. Otherwise, the release is delayed until outstanding issues are resolved.

Industry 2: Pharmaceutical Logistics – Asterion Biologics Distribution

Process: The workflow manages the transportation of vaccine lot ABX-2048 from Johannesburg to Cape Town. It includes order authorisation, stock verification, cold-chain packaging, carrier allocation, regulatory documentation, in-transit monitoring, delivery verification and performance reporting.

Participants: Pharmaceutical Order Coordinator, Responsible Pharmacist, inventory controllers, warehouse operators, packaging technicians, transport planners, carrier controllers, customs specialists, QA pharmacists, telemetry analysts and compliance managers.

Branching: The workflow uses parallel execution to monitor IoT temperature readings and carrier tracking during transportation. Approval checkpoints ensure that stock, packaging and final delivery meet the required conditions. Events such as temperature excursions can trigger separate exception workflows.

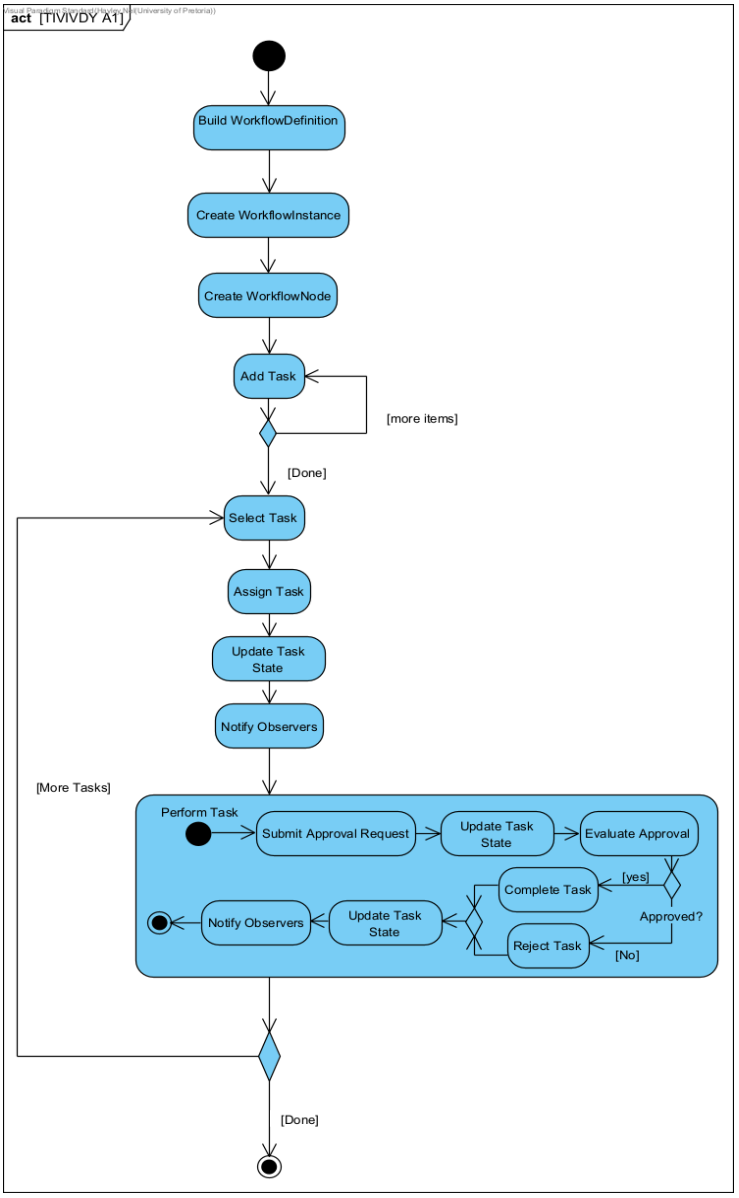
Failure paths: Temperature excursions trigger an investigation and quarantine process, followed by an authorised quality decision. Customs holds require corrected documentation,

while carrier failures and delivery delays require alternative transport arrangements and escalation.

Outcomes: The shipment is successfully delivered and accepted if all quality, documentation and delivery requirements are met. If a temperature excursion makes the stock unsuitable, it is rejected and a replacement shipment is arranged. Unresolved issues are escalated for further investigation.

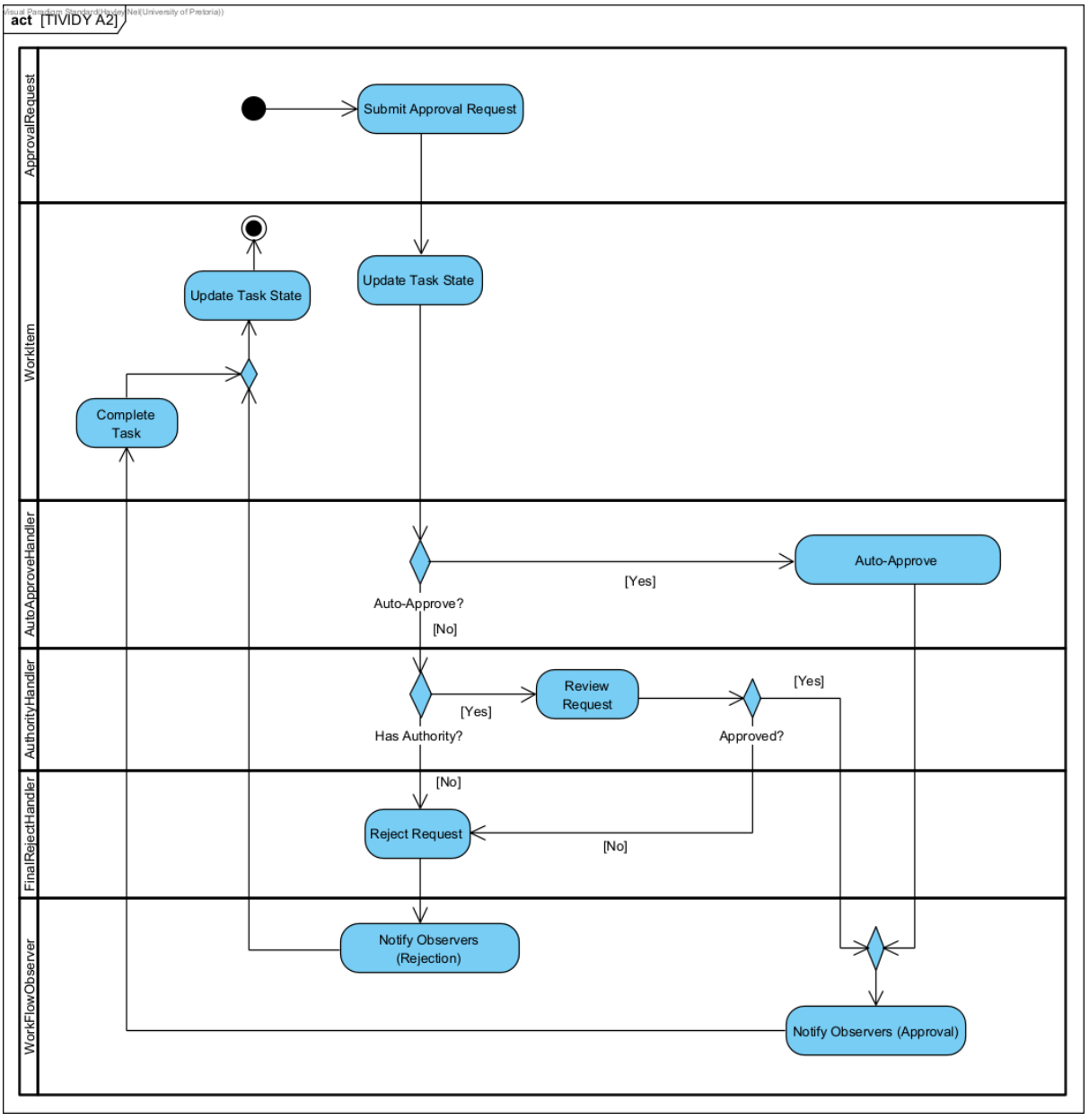
Activity Diagram 1:

Generic Workflow from creation to completion



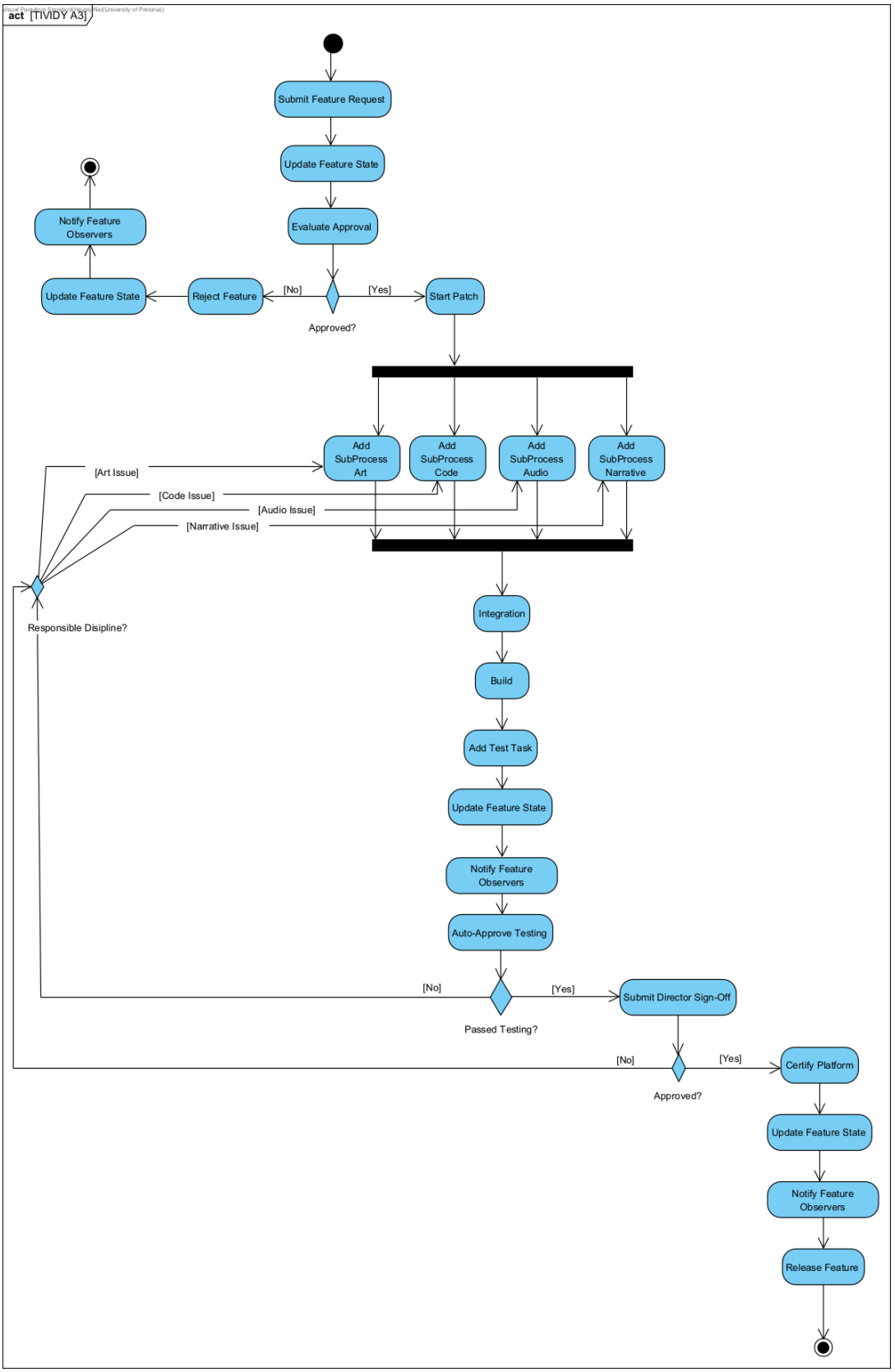
Activity Diagram 2:

TIVIDY Approval Process



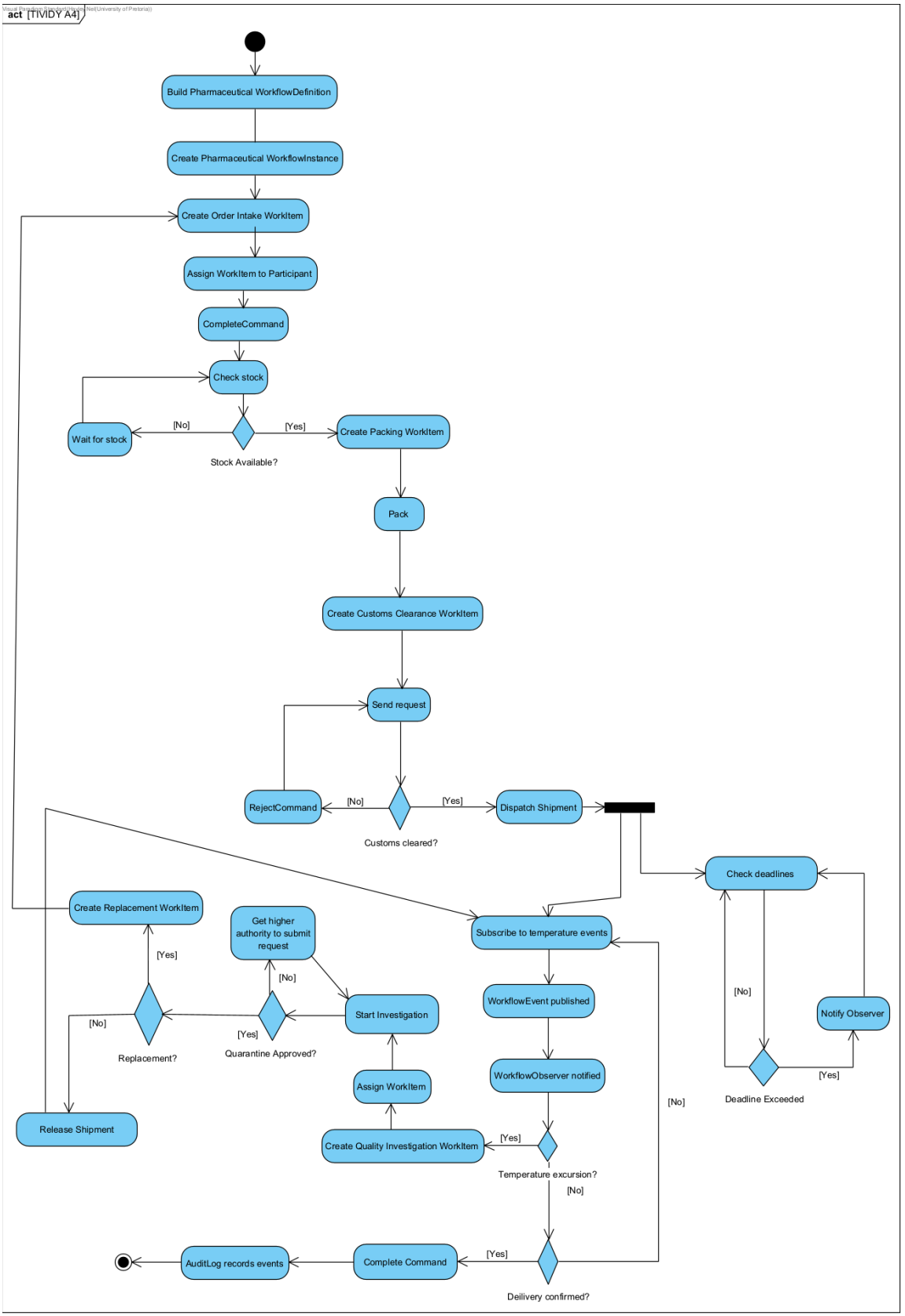
Activity Diagram 3:

Scenario A Workflow: Game Development

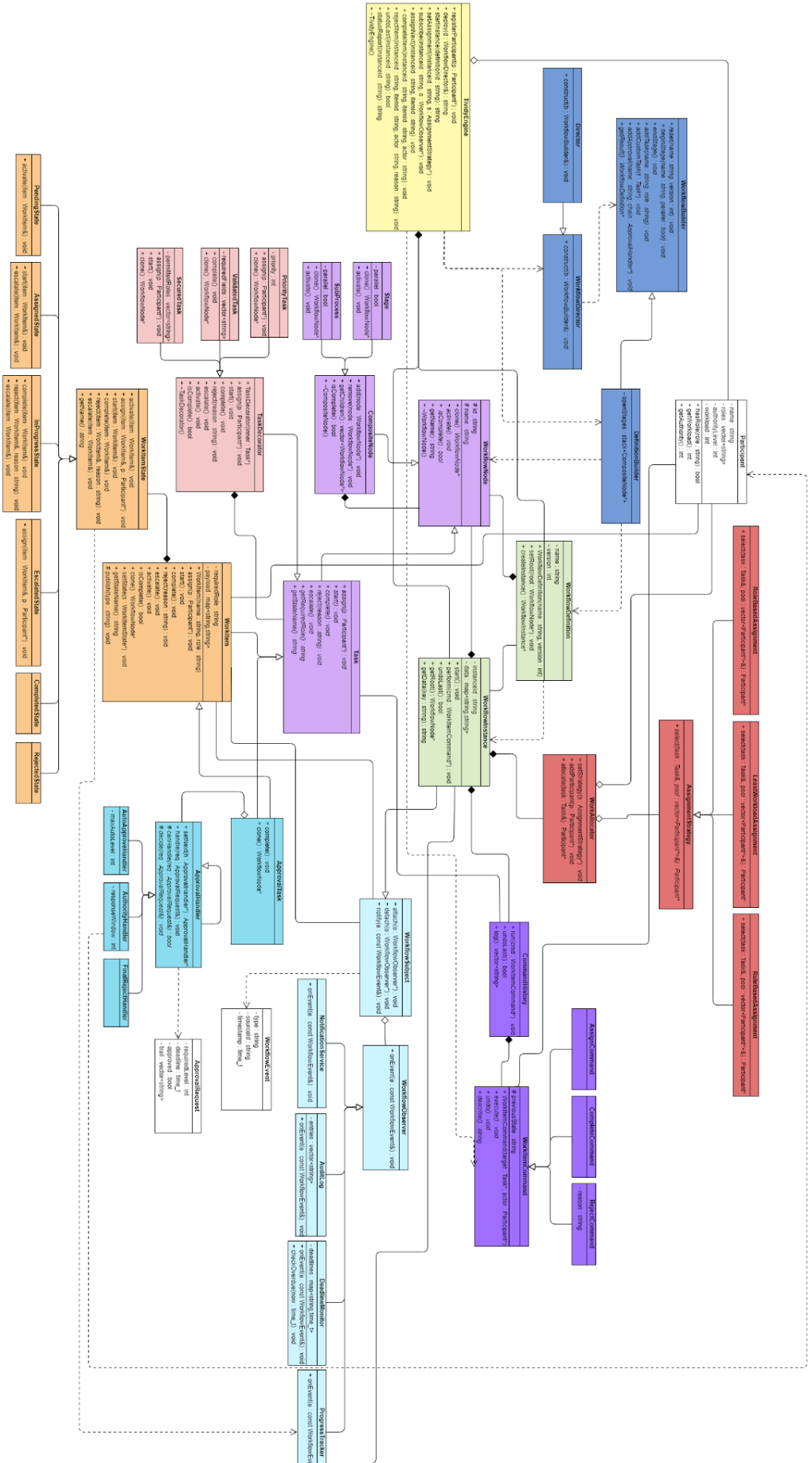


Activity Diagram 4:

Scenario B Workflow: Pharmaceutical Logistics

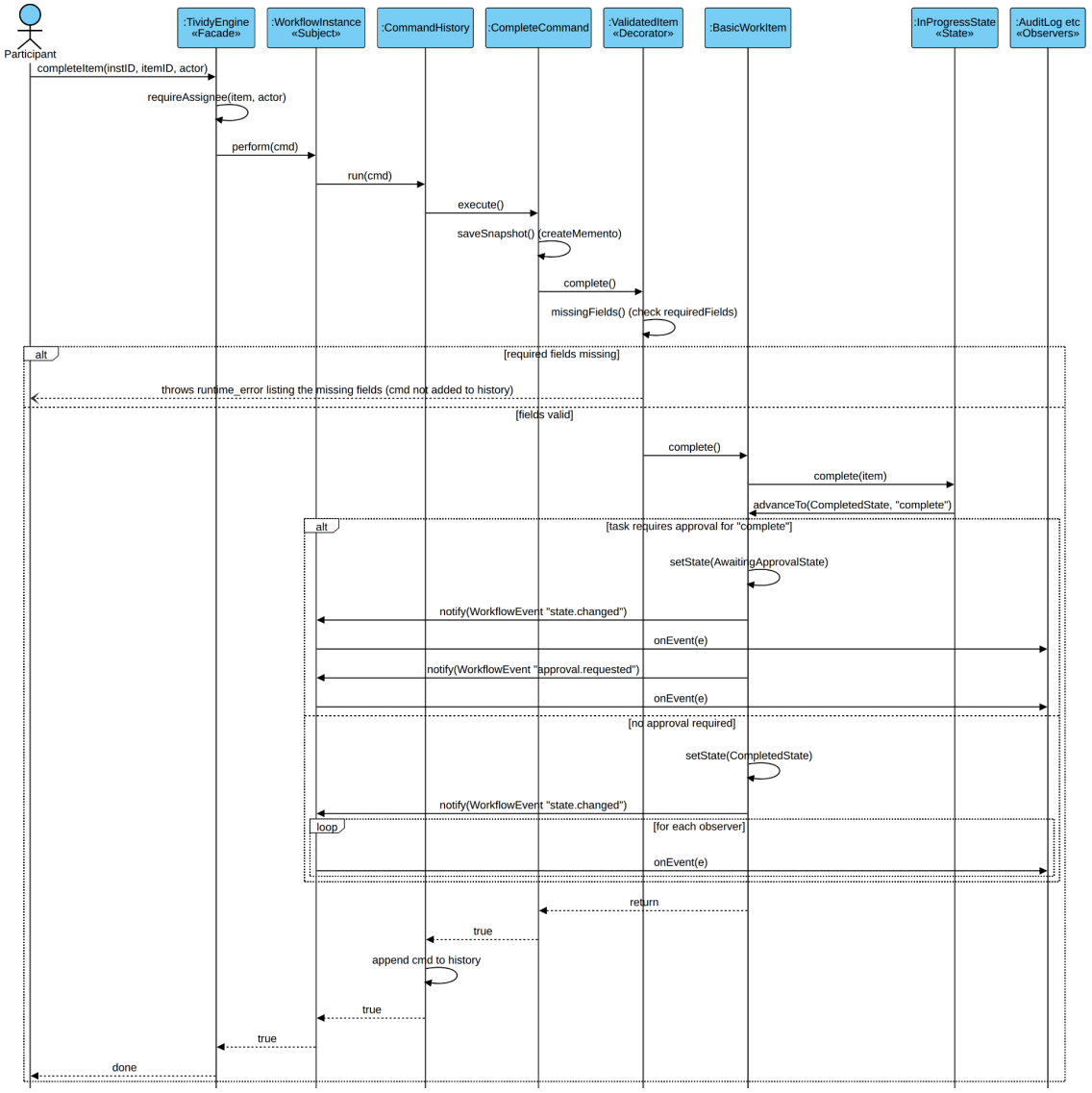


UML:



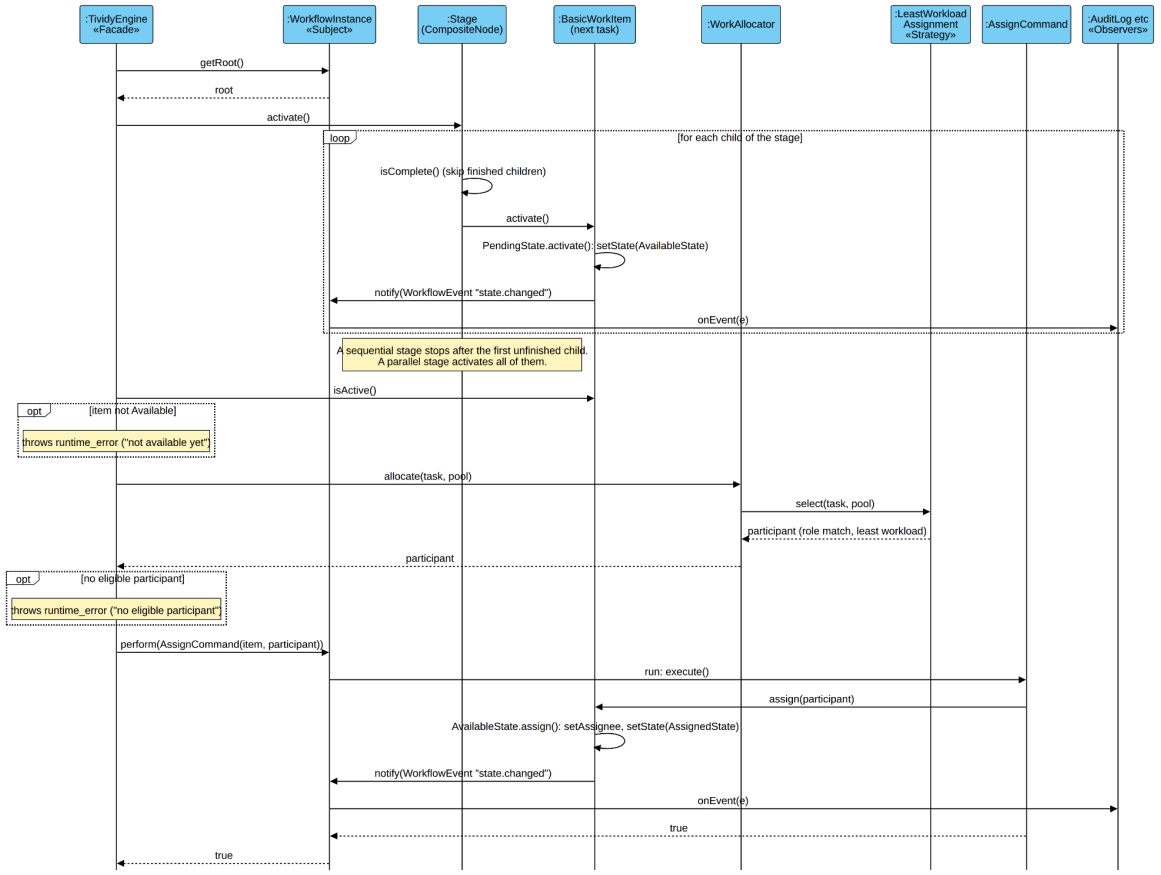
Sequence Diagram 1:

Completing a work item.



Sequence Diagram 2:

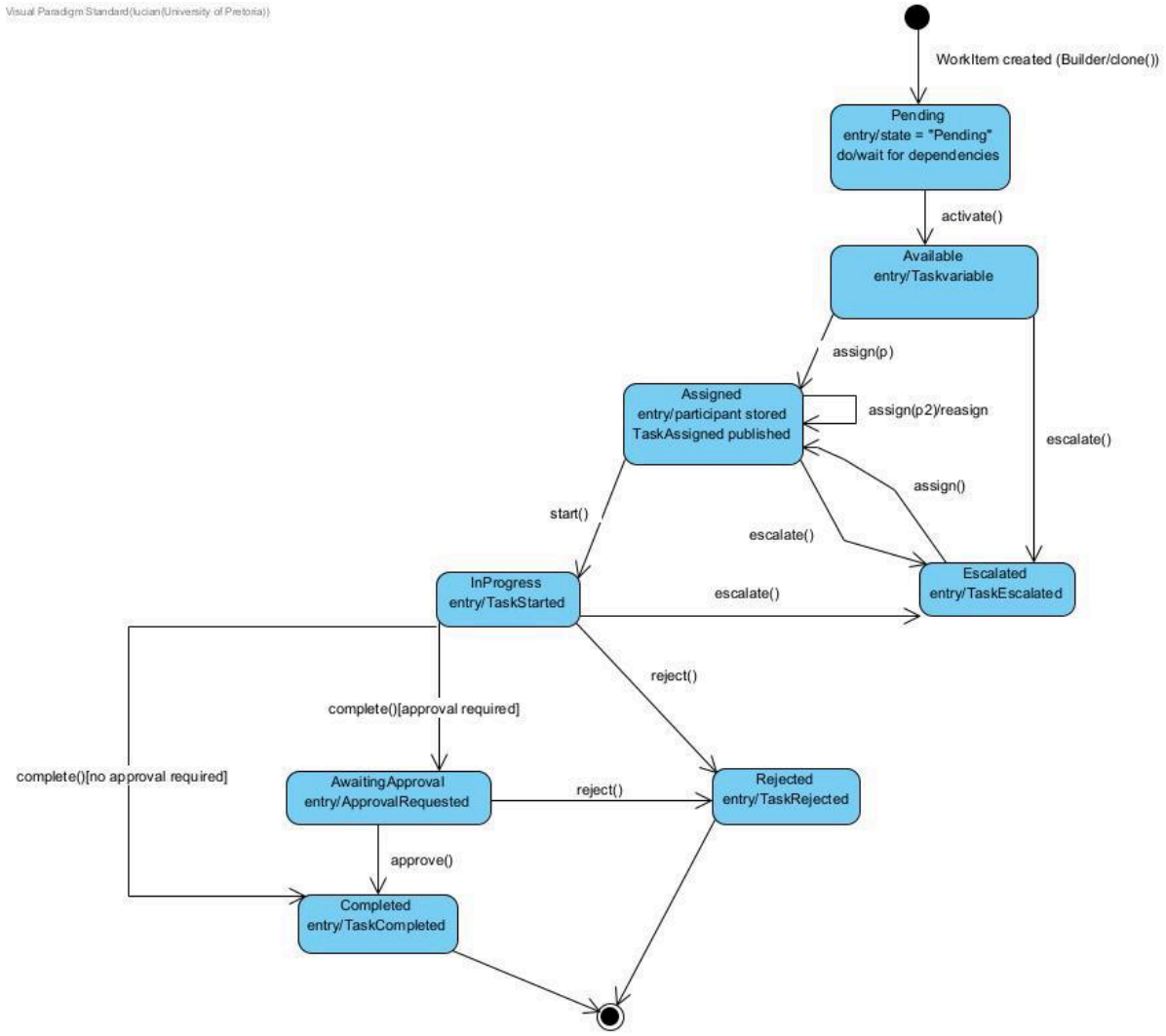
Determining what happens next.



State Diagram 1:

A work item has eight states. It starts in **Pending** and moves to **Available** when it is activated, then to **Assigned** once a participant is chosen. From **InProgress** it completes directly unless the task requires approval, in which case it waits in **AwaitingApproval** until approve() completes it or reject() rejects it. An Escalated item can be assigned again. **Completed** and **Rejected** are final.

WorkItemState.



Pattern Roles & Justifications

Composite -

Problem: Workflows contain stages, tasks and sub-processes, nested arbitrarily

Using the Composite design pattern, we give the workflow system a structured hierarchy. WorkflowNode, CompositeNode and WorkItem as Component, Composite and leaf. Two additional “concrete” composite classes are made: Stage and SubProcess to further define actions/ rules specific to the situation.

Decorator -

Problem: Tasks gain validation, priority, audit or security without new subclasses

We use the Decorator design pattern to add more functionality and information to tasks without editing the task itself. We achieve this with the classes WorkItem, WorkitemDecorator and concrete decorators. Since, in our design, WorkItem acts as its own component class which has different children.

State -

Problem: Work item lifecycle: valid operations depend on state

Using the State design pattern we allow the tasks to have defined states which can be reacted to. This allows additional states to be added, existing states to be edited and maintained. By removing this from the Task class itself, it allows more versatility when expanding upon the design.

Chain of Responsibility -

Problem: Multi-level approval and escalation

Using the Chain of Responsibility design pattern allows a line of approval to be created, maintained and edited. By doing so, any additions can be added easily and each stage can set new rules and requirements depending on the task.

Observer -

Problem: Many parties react to the same workflow event

By using the Observer design pattern, we create a system which allows changes/events in workflow instances to be reacted to. A workflow instance inherits from (is a) subject in the Observer design pattern. Any updates made will be sent directly to any number of observers attached such as AuditLog, NotificationService, etc.

Command -

Problem: Actions on work items, recorded for audit and undo

By using the Command design pattern, we turn each action on a work item into its own object that can be executed and reversed. A history keeps every executed command, giving a full audit trail and allowing undo. This prevents changes from being applied untracked or without a way to revert them.

Builder -

Problem: Constructing a valid definition tree step by step

By using the Builder design pattern, we define the set definition of a valid tree and the steps to create one. This prevents broken or invalid trees from being created.

Strategy -

Problem: Assignment and routing rules differ per organisation

By using the Strategy design pattern we describe and define processes for different assignment and routing rules, allowing a specific rule to change during runtime, where necessary. By using this design pattern, we allow additional strategies to be created and the current strategies to be edited and maintained without interfering with existing code. This allows less generic implementation for future uses.

Facade -

Problem: Simple entry point for scenarios/main

By using the Facade design pattern, we abstract the entire design further for the user's accessibility. This allows a complicated system to have a simple entry point with defines operations, without having to understand how every class connects with one another.

Prototype -

Problem: Many independent instances from one definition

By using the Prototype design pattern to easily produce independent copies of workflows without complicated creations from scratch. This prevents unnecessary overhead on the user end, allowing creation without fully understanding the underlying structure.

Memento-

Problem: Saving and restoring previous WorkItem states

By using the Memento design pattern, we allow WorkItems to save snapshots of their previous states and restore them when necessary. This provides undo functionality without exposing the internal details of the WorkItem. The WorkItemMemento stores the previous state, while the WorkItem is responsible for creating and restoring these snapshots. This allows changes to be reversed when needed.

Iterator -

Problem: Traversing a nested workflow tree without exposing its structure

By using the Iterator design pattern, we can step through every stage and work item in a workflow without the caller needing to know how the tree is built or whether a node is a Stage, SubProcess or WorkItem. WorkflowNode acts as the aggregate through createIterator(), with WorkflowIterator as the iterator and DepthFirstIterator and NullIterator as the concrete iterators. Stages return a DepthFirstIterator, which visits every node below them in order, while work items return a NullIterator since they have nothing below them. This keeps the traversal logic in one place, so features can use the same simple loop on a whole workflow, a single stage or one item.

Changes after Practical 6:

Addition of:
Memento and Iterator

Implementation Explanation and Important requirements to classes, patterns and tests:

Observer: Decouples workflow state changes from services which react to them

Subject: WorkflowSubject
Concrete Subject: WorkflowInstance
Observer: WorkflowObserver
Concrete Observers: NotificationService, AuditLog, DeadlineMonitor, ApprovalService
Other participating classes: WorkflowEvent

Workflow Event defines everything to do with an event in a workflow such as:

- Type of event - format of “DeadlineSet”, “ApprovalApproved”, “ApprovalRequested”, “ApprovalRejected”, “WorkItemCompleted”
- It’s associated WorkItem and WorkflowInstance
- A short description (detail)
- The time the event was created
- An optional deadline

Events are pushed into a FIFO Queue & dispatching bool flag ensures that the dispatch loop processes events sequentially. SO if an observer triggers a new workflow event while handling a current one, it won’t crash

Push Model: event occurs → subject packages context into WorkflowEvent → pushed to observers (onEvent(event)) so observers do not have to query the workflow instance for changes

When a state changes → workflow instance calls PublishEvent → details into WorkflowEvent → triggers notify()

Observers:
AuditLog - Records formatted strings of every event into internal vector
NotificationService - Prints updates
DeadlineMonitor - maintains a map of WorkItems and their deadlines. It adds to the map on event type setDeadline and removes it on WorkItemCompleted or ApprovalRejected
ApprovalService - tracks and logs approval requests, approvals and rejections

Chain of Responsibility: A line of approval for ApprovalRequest Objects

Handler: ApprovalHandler
Concrete Handlers: AutoApproveHandler, AuthorityHandler, FinalRejectHandler
Other participating classes: ApprovalRequest, Participant

AutoApprove → Authority → FinalReject
AutoApprove= Approves low-risk reqs (the request’s required level <= the max autolevel)

Authority= Role-based validation (the requester's authority level must be equal to or higher than the request's required level)
FinalReject= Rejects remaining requests

Strategy: Assignment Strategies

Context: WorkAllocator
Strategy: AssignmentStrategy
ConcreteStrategies: RoleBasedAssignment, LeastWorkloadAssignment, RuleBasedAssignment
Other participating classes: Participant, Task

WorkAllocator hold the active strategy, coordinates the allocation process and increments the workload of the participant it allocates

LeastWorkload= returns the participant with the lowest current workload. If more than one participant has the same lowest number of workloads then it returns the first one it found
RoleBased= returns the first participant who possesses the role required by the task
RuleBased= returns the participant who matches the required role and have a workload below 5 (this is just the rule I made up, we can change it)

If there are no participants in the pool or no participants match the requirements, they return null
WorkAllocator's allocate() returns null if it is unable to allocate a participant

Builder: construct a WorkflowDefinition, which is a nested tree of stages, tasks and approval rules

The components are:

WorkflowBuilder: the **abstract builder**, defining the steps (reset, beginStage, addTask, addCustomItem, requireApproval, endStage, getResult).
DefinitionBuilder: the **concrete builder**. It creates the Stage and BasicWorkItem nodes and the Task specs, and keeps track of which stages are open.
WorkflowDefinition: the **product**. It holds the name, version, tree root, task specs and approval chains.
Director and WorkflowDirector: the **director**. It reads the script file and calls the builder in order.
Script commands: WorkflowCommand, StageCommand, EndCommand, TaskCommand, ApproveLineCommand and DoneCommand, one per script word, sharing a DirectorContext.

This keeps construction separate from the finished object, so the director never touches the tree directly and the builder can validate the structure before handing it over. We assumed:

reset() is called first.
requireApproval directly follows the addTask it applies to.
Every stage is closed before getResult().

Approval chains are registered with the director before an approve line names them.
Task specs are shared between the definition and its work items.
Errors are thrown as exceptions, which the director catches and reports with a line number.

State: for the work item lifecycle

The components are:

BasicWorkItem: the **context**. It holds the current state and delegates assign, start, complete, reject, escalate and approve to it. It also offers setState and advanceTo for the states to use.
WorkItemState: the **abstract state**. Every transition is rejected by default, so each concrete state only overrides the moves it allows.
Eight **concrete states**: PendingState, AvailableState, AssignedState, InProgressState, AwaitingApprovalState, EscalatedState, CompletedState and RejectedState.
WorkItemMemento: it stores a clone of the current state, so undo can restore an item.

This replaces one large if/switch with a small class per state. Each state decides which transition is legal and which state comes next. We assumed:

Illegal transitions throw std::logic_error.
Completed and Rejected are terminal.
Re-activating an item has no effect.
Only completion can require approval, through advanceTo, which wraps the target state in AwaitingApprovalState.
Rejection skips approval.
States clone themselves so the memento can snapshot and restore an item.

Prototype: to create running workflow instances from a workflow definition

Each instance gets its own deep copy of the definition's tree, so many cases can run at once without affecting each other or the template. The components are:

- WorkflowNode: the **prototype**. It declares clone(), which every node in the tree must implement.
- **Concrete prototypes**:
    - Stage and SubProcess copy themselves, then clone each of their children into the copy (CompositeNode::cloneChildrenInto).
    - BasicWorkItem copies its current state, assignee and data.
    - PriorityItem, ValidatedItem and SecuredItem clone the item they wrap, so the decorators are kept.
- WorkItemState: each of the eight states also has clone(), so a work item can copy whatever state it is in without a switch.
- WorkflowDefinition: the **client**. createInstance() calls clone() on the root and gives the copy to a new WorkflowInstance.

Iterator**:** to walk a workflow tree without the caller needing to know how it is built, or whether a node is a Stage, a SubProcess or a work item

The components are:

- WorkflowIterator: the **abstract iterator**, defining first(), next(), current() and isDone().
- DepthFirstIterator: the **concrete iterator** for stages. It uses a stack to visit every node below its starting node, depth-first, top to bottom.
- NullIterator: the **concrete iterator** for work items. A work item has nothing below it, so this iterator is finished straight away.
- WorkflowNode: the **aggregate**. It declares createIterator().
- **Concrete aggregates**: CompositeNode (and so Stage and SubProcess) returns a DepthFirstIterator, and WorkItem returns a NullIterator.

Command: to make actions by participants have a structured way of interacting with the workflow

This avoids having to handle the complicated tasks directly. We also store these commands in a history, so that commands can be reverted.

- WorkItemCommand: **Abstract command**, defining execute(), undo(), redo(), describe()
- **Concrete Commands**:
    - AssignCommand
    - CompleteCommand
    - RejectCommand
    - ApproveCommand
    - StartCommand
- CommandHistory: Used as a buffer between commands and Workflow. In charge of storing previously executed commands and restoring/undoing changes.

Memento: to save and restore states of WorkItems

It provides revert functionality with states.

- WorkItemMemento: Stores a snapshot of the previous state, so that we can undo this change later
- WorkItemState: What is captured by the memento
- WorkItem: The class in charge of storing and restoring Memento

Composite: is used to represent any workflow as a nested “tree” of stages, sub-processes and work items

The composite structure as follows:

Component -> Stage & SubProcess. Parallel stage activates all children at once (fork), sequential stage activates all children sequentially. SubProcess waits for an inner node to finish before the outer one does. Furthermore, WorkflowNode is the abstract component.

Composite -> CompositeNode. Maintains a list of its children, and provides the add and remove functionality.

Leaf -> WorkItem. No children, so calling child(i) throws.

This structure allows the system to activate, bind, clone and check completion for a single task or an entire workflow without type checking. Only composites have the ability to add children, such as to safeguard against giving leaves children by accident.

Decorator: is used to add priority, validation and/or security rules to a work item without subclassing for every possible combination

Structure is as follows:

AbstractDecorator -> WorkitemDecorator. A Workitem and also owns the workitem it wraps.

ConcreteDecorator -> PriorityItem, SecuredItem, ValidatedItem. Adds complexity/functionality to a workItem. Priority adds a priority value, SecuredItem restricts the use of a workitem to certain roles, and validatedItem refuses to complete an item until every necessary field is filled.

Component (shared with Composite) -> BasicWorkItem. Holds the necessary fields of TaskSpec, state, assignee and payload.

We did assume that every Decorator takes ownership of every item it wraps, and thus destroys it after. As well as a refused operation does not change the item state, just throws.

Adapter is used to hide the underlying complexity of tividy from the client without sacrificing functionality

Adapter -> TividyEngine

Design Decision History:

- Removal of Adapter:

Every adaptee was **scenario-specific**, so the team could not keep the integration layer generic without it being shaped by one scenario's external systems. The Facade solves a problem that exists for every scenario:
Client code had to work directly with builders, commands, states and allocators to run a workflow. Through TividyEngine, scenario code uses the engine without depending on its internals, which keeps the scenario separate from the generic system. External systems are now described in the scenario but are outside TIVIDY's design scope. They can be reintroduced if an integration requirement is added.

- Putting the pipeline in between Dev and Main rather than the feature branches and Dev

Feature branches needed classes from different patterns in different branches and so couldn't run without them. Therefore we opted to rather pull from Dev and resolve conflicts and then merge to Dev and do testing in a separate branch with all the necessary classes.

- Builder: script commands and the addition of an extra command design pattern

The first version of WorkflowDirector wasn't generic enough so we decided to use a textfile as input when constructing the scenarios since it would be more practical than manually typing out a scenario on each execution. The first iteration of this new WorkflowDirector only understood stage and task lines, this was too little context for a proper scenario to be built. To help with the execution of this we added a new command design pattern where every script keyword is its own small command class. The approve command was one addition. The line approve | trigger | chain attaches a named approval chain to the task declared directly before it. Approval rules therefore live in the scenario. The builder also refuses to build a half-built workflow, an unknown command or syntax would be reported with its line number and the directors construct will return false to the facade.

Build/run/test instructions:

````
Install on Ubuntu/Debian:

```bash
sudo apt update
sudo apt install build-essential make libncurses-dev
```

**Docker**
- Docker Engine or Docker Desktop

## Build

### Using Make

Build from the project root:

```bash
make
```

Run the default demo:
````

```bash
./main
```

Run with a workflow file:

```bash
./main test.txt
```

Clean build files:

```bash
make clean
```

Using Docker

Build the image:

```bash
docker build -t tividy .
```

Run the default demo:

```bash
docker run --rm -it tividy
```

Run with a custom workflow file:

```bash
docker run --rm -it tividy /app/workflows/workflow_file.txt
```
To Test:
Download the doctest.h file and place it into /tests
```bash
cd tests
g++ -std=c++11 -Wall -Wextra -I../include ../src/*.cpp test_pattern.cpp
-o test_pattern_runner
```

```
Replace "pattern" with the pattern of the test file you would like to
run
```bash
./test_pattern_runner
```
```

Bonus marks:

- CI/CD pipeline
- Dockerised application
    - Dockerfile which enables the application to be built and run using docker. Builds the image on top of a debian environment.

Team Contribution:

Matthew - Initial pattern justification, Docker, Command, Memento, TUI and Scenarios
Ruben - Workflow research, CI/CD pipeline, Facade, Composite and Decorator
Keagan - Initial UML, scenario design, prototype and Iterator
Hayley - Activity Diagrams, Observer, Chain of Responsibility, Strategy, PDF Report and Demo Powerpoint
Lucian - Sequence Diagrams, State Diagram, Builder, State, Doxygen and UML fixes.

All members contributed unit testing using Doctest